AI Usage Report

Course:	Software For Mobile Devices
Assignment:	Assignment 1 — AI-Assisted Application Development (University Student Portal)
Student Name:	Manahil Fatima
Registration No.:	23i-3053
Date:	3rd Oct 2026

---

1. AI Tool(s) Used

Claude (Anthropic) used throughout the project via a conversational chat interface, for planning, code generation, debugging, and iterative feature development.

2. Purpose of AI Usage

AI was used for the following purposes during development:

- Planning the overall file/folder structure for a React Native project with no navigation library
- Generating the initial code for reusable components, screens, and data files, one file at a time
- Debugging real runtime errors (missing imports, missing dependencies, broken asset paths) by reading error stack traces
- Designing and implementing two original features beyond the base requirements: a What-If Attendance Simulator and a weighted Grade Predictor with absolute-marks calculation
- Reviewing the application against the assignment rubric to identify gaps and deciding how to close them
- Improving the visual design system (color palette, typography) to move away from a generic default look

3. Important Prompts Used

Prompt 1:

"We will build 5 screens for the FLEX: Login, Dashboard, Courses, Attendance, and Course Details... tell me step by step which file to replace... which code to write first?"

Prompt 2:

"How can a student mark the attendance" followed by "I like it and want to implement this" (regarding an attendance projection / what-if simulator idea) used to design and build the What-If Attendance Simulator feature from scratch, including the underlying calculation helpers.

Prompt 3:
"In our university there is an absolute marking like the quiz holds a weightage of 5 absolutes... students have to manually calculate the number of absolutes they get on the basis of some marks and on that grades is calculated" — used to design the weighted Grade Predictor feature, including per-component absolute-marks breakdown and the 'what score do I need for an A / to pass' logic.

Prompt 4:
"Check if my assignment completes all the requirements" used to self-audit the finished app against the official assignment rubric section by section and identify remaining gaps.

4. AI-Generated Output

The AI generated the source code for the project in incremental steps: the data layer logic (student/course objects and calculation helper functions), six reusable components (Custom Button, Input Field, Progress Bar, Header, Course Card, Star Rating), five screens files structure (Login, Dashboard, Courses, Attendance, Course Details), and the top-level App.js that holds the current screen and the shared course data in state. It also generated the logic for two original features — the What-If Attendance Simulator (projected attendance based on hypothetical future classes) and the weighted Grade Predictor (absolute marks per component, predicted letter grade, and the score needed on remaining components to pass or get an A) — along with the Course Feedback feature (star-rating submission form with live average-rating recalculation).

5. Changes Made by Me

I ran every file in Expo after each step rather than accepting code blindly and caught several real issues this way: a missing react-native-chart-kit dependency that broke the Dashboard screen, a broken app.json asset path causing a Metro bundler error, and two incomplete import statements (StarRating and TextInput) that caused render errors after a multi-file edit. I also changed the roll-number validation format to match my actual university format (23I-3054 instead of the AI's initial placeholder format), removed a Mark Attendance (Present/Absent) feature I decided didn't fit the scope after reviewing it, and redirected the AI's first feature suggestion (an admin-style 'Add Course' feature) toward a student-perspective feature (Course Feedback) instead, since Add Course did not reflect something an actual student would do.

6. My Understanding

The application's data (the courses array) lives in one place — App.js — using useState, instead of each screen importing a static file independently. This is what is meant by 'lifting state up': multiple screens (Dashboard, Courses, Attendance, Course Details) all need to see the same, up-to-date course data, so the data has to live in their common parent and be passed down as props. When something changes — for example, a student submitting course feedback — the handler function in App.js never edits the existing array directly. Instead it uses .map() to build a brand-new array where only the matching course object is replaced with an updated copy (using the spread operator). React detects that the array reference has changed and re-renders every screen that depends on it. This immutable-update pattern is why a new feedback entry, once submitted, is immediately reflected in the average rating and review

count without needing to navigate away or refresh the screen.

The Grade Predictor works by storing raw obtained/total marks per component (quiz, assignment, project, mid1, mid2, final) rather than a single precomputed percentage. Each component's percentage is calculated on the fly and multiplied by its fixed weightage out of 100 to get the 'absolute marks' earned for that component; these are summed with reduce()-style accumulation to get the total absolute score so far. A component with total: 0 is treated as 'not held yet' and excluded from the score but included in the remaining weight, which is what lets the app calculate the average percentage needed on the remaining components to reach a target score (85 for an A, 50 to pass), including correctly detecting when a target is already secured or no longer achievable.

7. Verification and Testing

Every file was tested by running the app in Expo (npx expo start) immediately after it was added or edited, rather than writing multiple files before testing anything. This surfaced several real errors: an 'Unable to resolve module react-native-chart-kit' bundling error, caused by the package not being installed yet — fixed by running npx expo install react-native-chart-kit react-native-svg and clearing the Metro cache. A 'Property StarRating doesn't exist' render error and a 'Property TextInput doesn't exist' render error, both caused by incomplete import statements after a multi-section file edit — fixed by checking the top of the affected file and restoring the full import line. An asset-resolution error referencing app.json's icon path, which was a configuration issue unrelated to the app's React code. After each fix, the affected screen was reloaded and manually interacted with (submitting the login form with both valid and invalid data, using the attendance simulator stepper, submitting course feedback with and without a rating, and viewing a course with and without existing feedback) to confirm the feature behaved correctly in every state, not just the happy path.

8. Reflection

Working file-by-file instead of generating the whole app at once made it much easier to actually understand what each piece was doing and to catch problems early, rather than debugging a large block of unfamiliar code all at once. I learned that AI-generated code is only as good as the testing applied to it several real bugs only surfaced because I ran the app after every change rather than trusting that it would work. I also learned that closely reading the assignment rubric against a finished feature set (rather than just asking 'does it work') is what revealed a genuine gap — the Data-Driven and Dynamic Behavior requirement — that the original feature set did not fully satisfy, which led to deliberately adding the Course Feedback feature. Overall, the experience reinforced that AI is most useful as a fast first-draft generator and a debugging partner, but the responsibility for understanding, testing, and defending every part of the submitted code remains mine.

Student Declaration

I confirm that I have used AI tools only as a development assistant and that I understand the code submitted as part of this assignment. I can explain and demonstrate the functionality of my application.

Student Name: Manahil Fatima

Signature: **ManahilFatima**

Date: **3rd oct 2026**